

COMPTE RENDU TP 2 RÉSEAUX

1)

Entête Ether (0x0001) Hardware type : Ethernet (1)

Données ARP (0x0806)

Protocole type : IPv4 (0x0800)

Ce qui permet d'identifier un paquet comme étant de type ARP, c'est :

- Le champ Ethernet, avec la valeur 0x0806.
- Les champs spécifiques du message ARP (Hardware Type, Protocol Type, etc.) qui suivent l'en-tête Ethernet.

2) Le paquet ARP se situe dans la trame ethernet, dans la section DATA.

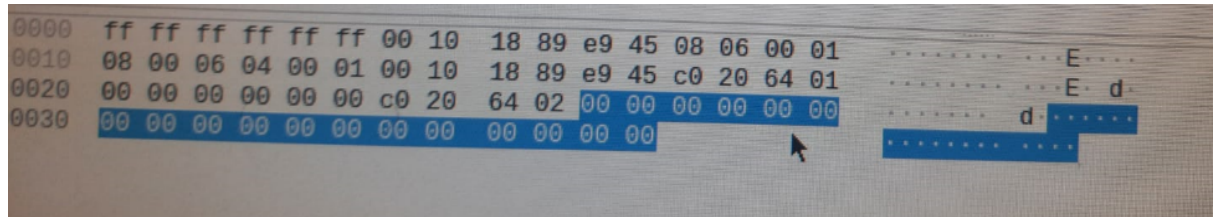
```

▼ Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: Broadcom_89:e9:45 (00:10:18:89:e9:45)
  Sender IP address: 192.32.100.1
  Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
  Target IP address: 192.32.100.2
  
```

3)

Hardware type (2)	Type de matériel sur le réseau local
Protocol type (2)	Type de protocole, ici ethernet = 0x0806
Hardware size (1)	Taille de l'adresse du matériel
Protocol size (1)	Taille de l'adresse du protocole, 4 octets pour Ipv4
Opcode (2)	Spécifie la nature du message ARP
Sender MAC address (6)	Adresse mac de la machine qui envoie le paquet
Sender IP address (4)	Adresse IP de la machine qui envoie le paquet
Target MAC address (6)	Adresse mac de la machine qu'on cherche à joindre
Target IP address (4)	Adresse IP de la machine qu'on cherche à joindre

4)



Le niveau Ethernet connaît la taille de la trame grâce à deux mécanismes :

- Champ longueur ou taille implicite
- Minimum et maximum de la taille :

Une trame Ethernet doit respecter une longueur minimale de 64 octets et maximale de 1518 octets. Si la taille minimale n'a pas été atteinte, des octets de bourrage (padding) sont ajoutés.

5) La couche ethernet transmet à la couche ARP l'adresse MAC de la source et la destination du paquet . Comme dit précédemment, si la charge utile de données est inférieure à 64 octets, un padding est ajouté à l'envoi. Ethernet connaît la taille envoyée en amont mais, à la réception, le bourrage est retiré puisque traité dans la couche physique, il ne connaît donc pas l'existence du bourrage.

6) attendre(événement)

- si événement est "Question sur adresse internet" (requête interne de IP vers ARP)
 - si adresse==mon adresse
 - répondre puis ajouter adresse dans table ARP pour 1200 secondes
 - sinon
 - ne rien faire

fin si

- si événement est expiration du timer d'effacement associée à une entrée
 - enlever_de_table(entrée)

finsi

- si événement est réception requête (ARP Request)
 - si requête m'est adressée OU si j'ai la réponse dans ma table ARP
 - répondre avec l'adresse ethernet demandée

finsi

finsi

- si événement est Announcement
 - modifier la table en conséquent

finsi

- si événement est ARP Gratuit
 - mettre à jour la table ARP en fonction des informations fournies dans le paquet

finsi

fin tantque

7) B apparaît dans la table ARP de A parce qu'avant d'envoyer les paquets ICMP (pour le ping), A doit connaître l'adresse de B. Si cette adresse n'est pas déjà présente dans la table ARP de A, la machine A envoie une requête ARP à tout le monde pour demander l'adresse de B. Lorsqu'elle reçoit une réponse de B, elle met à jour sa table ARP en ajoutant l'adresse de B. Pour l'apparition de A dans la table de B, B apprend l'adresse MAC de A de manière passive lors de la réception de la requête ARP envoyée par A, B voit l'adresse de A et met ensuite à jour sa table ARP pour y stocker l'adresse de A.

8)

Apply a display filter ... <Ctrl-/>

	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.32.100.1	192.32.100.2	ICMP	98	Echo (p
2	0.000048050	Broadcom_89:ea:39	Broadcast	ARP	42	Who has
3	0.000398528	Broadcom_89:ea:45	Broadcom_89:ea:39	ARP	60	192.32.100.2
4	0.000408596	192.32.100.2	192.32.100.1	ICMP	98	Echo (p
5	1.018704510	192.32.100.1	192.32.100.2	ICMP	98	Echo (p
6	1.018744274	192.32.100.2	192.32.100.1	ICMP	98	Echo (p
7	2.026750684	192.32.100.1	192.32.100.2	ICMP	98	Echo (p
8	2.026782322	192.32.100.2	192.32.100.1	ICMP	98	Echo (p
9	3.089717839	192.32.100.1	192.32.100.2	ICMP	98	Echo (p
10	3.089743322	192.32.100.2	192.32.100.1	ICMP	98	Echo (p
11	4.100729234	192.32.100.1	192.32.100.2	ICMP	98	Echo (p
12	4.100756043	192.32.100.2	192.32.100.1	ICMP	98	Echo (p

Comme A connaît B, il ne fait pas de request ARP. Par contre B ne connaît plus A, quand il reçoit une ping request (ICMP) de A, B renvoie un paquet ARP pour obtenir l'adresse de A et pouvoir lui répondre.

9)

	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
2	1.009358065	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
3	2.031648204	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
4	3.040647993	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
5	4.050645986	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
6	5.063646654	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
7	6.072649052	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
8	7.081645465	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
9	8.090655315	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
10	9.091641801	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
11	10.107081561	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
12	11.120649673	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
13	12.124647379	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
14	13.125638816	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
15	14.150645707	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
16	15.196577862	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
17	16.202648346	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
18	17.253200465	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
19	18.289643323	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
20	19.353139766	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
21	20.394599953	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1
22	21.407850826	Broadcom_89:e9:45	Broadcast	ARP	42	Who has 192.32.100.2? Tell 192.32.100.1

C'est le timer de ping qui déclenche les émissions des paquets ARP request, la durée de ce timer de réémission est de à peu près 1 seconde.

10) Lors du changement d'adresse, la machine qui change d'adresse envoie un paquet ARP au broadcast. Les ARP gratuits sont des mécanismes utilisés pour maintenir et mettre à jour les tables ARP des machines sur le réseau local, assurant ainsi une connectivité réseau efficace et évitant les conflits d'adresses IP.

11)

Destination	Protocol	Length	Info
broadcast	ARP	60	ARP Announcement for 192.32.100.2
broadcom_89:...	ARP	60	Gratuitous ARP for 192.32.100.2 (Reply) (duplicate use of 192.32.100.2 detected!)

Le paquet ARP Announcement s'est chargé d'annoncer le changement d'adresse de A par celle de la machine B. Le message d'alerte émis par le paquet ARP Gratuit nous prévient que l'adresse est déjà utilisée par une autre machine.

12)

	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
2	1.012435904	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
3	2.017606211	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
4	3.018623836	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
5	4.025485515	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
6	5.046804940	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
7	6.047624088	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
8	7.054795373	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
9	8.068624386	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
10	9.069627549	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
11	10.133120093	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
12	11.171499194	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
13	12.235056505	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
14	13.268633682	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
15	14.315186275	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
16	15.378122289	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
17	16.386627517	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
18	17.397793819	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
19	18.454627325	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
20	19.455623343	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
21	20.458634557	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3
22	21.460628223	Broadcom_89:e...	Broadcast	ARP	42	Who has 192.32.100.1? Tell 192.32.100.3

des paquets à B, mais en réalité, les paquets sont transmis à A à cause de l'entrée ARP publiée, quand A reçoit les paquets ICMP destinés à B, A ne répond pas, donc C ne reçoit pas de reply et la connexion échoue.

13) Lors du ping on remarque que la requête et le reply ont la même valeur. Identifier compte le nombre de paquets et se met à jour à chaque nouvelle demande. Sequence Number permet de savoir à quel paquet appartient une requête et son reply.

14)

```
Identification: 0x869d (34461)
▶ 000. .... = Flags: 0x0
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 64
Protocol: ICMP (1)
Header Checksum: 0xabc6 [validation disabled]
[Header checksum status: Unverified]
Source Address: 192.32.100.1
Destination Address: 192.32.100.3
▼ Internet Control Message Protocol
Type: 8 (Echo (ping) request)
Code: 0
Checksum: 0xebdc [correct]
[Checksum Status: Good]
Identifier (BE): 3106 (0x0c22)
Identifier (LE): 8716 (0x220c)
Sequence Number (BE): 1 (0x0001)
Sequence Number (LE): 256 (0x0100)
[Response frame: 10]
▶ Data (56 bytes)

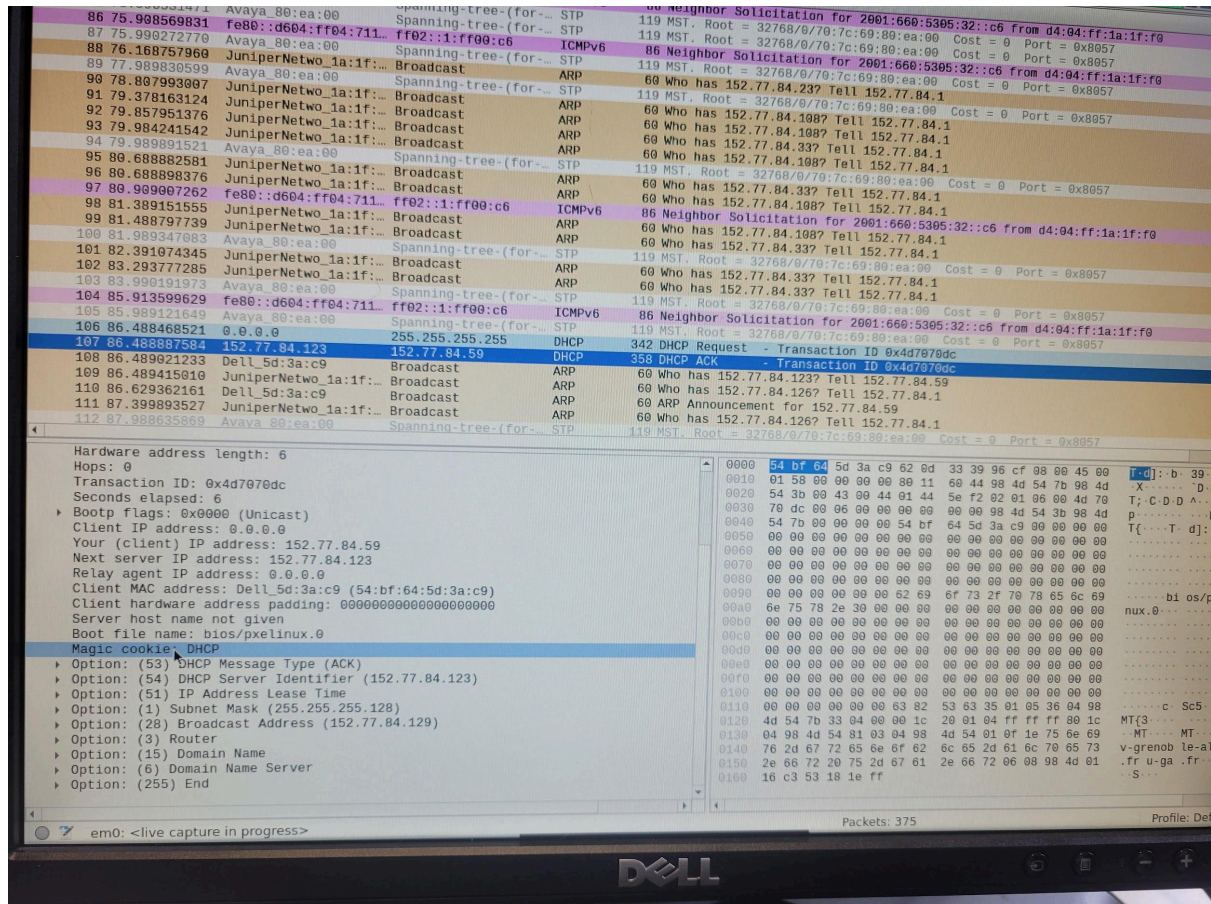
0000  00 10 18 89 e9 ee 00 10 18 89 e9 45 08 00 45 00  . . . . .E.E.
0010  00 54 86 9d 00 00 40 01 ab c6 c0 20 64 01 c0 20  .T. @ . . . d.
0020  64 03 08 00 eb dc 0c 22 00 01 00 00 00 00 00 00  d. . . . " . . .
0030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  . . . . .
0040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  . . . . .
0050  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  . . . . .
0060  00 00
```

Le checksum ICMP est utilisé pour détecter les erreurs dans les paquets ICMP. Il est calculé comme ça :

1. Diviser le message en mots de 16 bits (2 octets).
2. Additionner tous ces mots en utilisant l'addition sur 16 bits (somme en complément à un).
3. Si un dépassement se produit, l'ajouter au résultat.
4. Prendre le complément à un du résultat final.

Ce champ permet de vérifier la validité d'un paquet ICMP et peut détecter les erreurs de transmission comme les altérations accidentelles des données pendant leur transit sur le réseau; dues à des interférences électromagnétiques, des collisions sur le réseau, des erreurs de routage... Il ne peut cependant pas détecter d'erreurs multiples ou qui se compensent.

15)



Lors de l'exécution de la commande `dhcclient`, la machine envoie une requête pour détecter la présence de serveurs DHCP (dans ce cas, deux serveurs sont disponibles). Le premier serveur répond en attribuant une adresse IP et met à jour la configuration de l'interface `em0` via `ifconfig`. Cette modification entraîne une annonce ARP. Par la suite, le second serveur tente également d'attribuer une adresse IP, mais sa réponse est rejetée, la machine étant déjà connectée au premier serveur.

16)

- Identification permet de différencier chaque paquet grâce à un identifiant unique.
- Total Length représente la taille totale du paquet envoyé, incluant l'en-tête et les données.
- Fragment Offset indique si un paquet a été fragmenté et permet sa reconstruction à destination.
- Le flag MF (More Fragments) est positionné à 1 si le paquet est fragmenté et à 0 s'il ne l'est pas.

L'en-tête UDP est contenu uniquement dans le premier fragment, car il fait partie des premières données encapsulées dans le paquet d'origine. Les fragments suivants ne contiennent que des morceaux des données de l'application, sans en-tête UDP.

Les fragments sont envoyés en commençant par les bits de poids fort (c'est-à-dire les premiers octets du paquet d'origine), suivis des bits de poids faible (les octets suivants dans l'ordre du message). Chaque fragment contient un fragment offset pour indiquer sa position dans l'ensemble des données originales.

Pour retrouver les 4600 octets de données applicatives il faut reconstituer les fragments en fonction de leur fragment offset. En combinant tous les fragments et en éliminant les en-têtes IP intermédiaires, on obtient les données originales.

